

LLM Engineering Assignment - Building AI Agent

headref

An Evidence-Based Engineering Team Intelligence Agent with Multi-Source Ingestion and Deterministic Analyzing

Module: LLM Engineering

Date: October 25, 2026

Submitted By: - cclarke642 - W. A. S. Keshan (Group Leader)

- cclarke643 - K. Isiwara Didulani

Table of Contents

Executive Summary

1. Problem Description and Scenario

1.1 Operational Challenge

1.2 Scenario Scope and Solution Overview

2. Dual Data-Path Architecture

2.1 Write Path and Read Path Overview

2.2 Six-Stage Deterministic Agent Pipeline

3. Detailed Subsystem and Tool Designs

3.1 Write Path Services and Ingestion Engines

3.2 Agent-Facing Read Tools T-001–T-007

4. Shared Findings Engine and LLM Prompt Design

4.1 Deterministic Findings Engine

4.2 Prose Narrative Generation

5. Testing, Security, and Hardening

5.1 Write Path Security Gate

5.2 Read Path Evaluation Harness

6. Submission Deliverables and Individual Contribution Matrix

6.1 Project Deliverables

6.2 Individual Contribution Table

Appendix: AI Coding Assistant Disclosure

Headref

An Evidence-Based Engineering Team Intelligence Agent Built on a Local LLM with Ollama

EXECUTIVE SUMMARY

headref is an automated, AI-driven engineering intelligence system designed to give team leads and delivery managers instant, accurate operational insights into developer activities, blockers, and project risks. By capturing, normalizing, and correlating data from Jira and GitHub into a local canonical PostgreSQL database, headref answers daily workflow questions in seconds without direct developer status interruptions or expensive cloud API fees.

1. Problem Description & Scenario

1.1 The Operational Challenge

Engineering team leads and delivery managers frequently spend 20 to 40 minutes every morning attempting to answer basic operational questions before daily standups: 'What is a team member currently working on, is anything blocking them, and are there any delivery risks?' Today, answering this requires manually inspecting Jira boards, checking open GitHub pull requests, reviewing pending code reviews, and guessing how unlinked work items connect. Existing developer dashboards merely aggregate activity counters (commit counts, PR volumes), which fail to provide actionable context and often misrepresent progress.

1.2 Scenario Scope & Solution Overview

headref ingests read-only data from Jira and GitHub, normalizes it into a canonical relational schema, explicitly links Jira tickets to branches/PRs/commits, and evaluates three fixed operational questions per team member:

- **Current Work:** What is the developer currently working on?
- **Blockers:** What dependencies, review delays, or failing CI checks are obstructing progress?
- **Risks:** What overdue tasks, stale drafts, or discrepancies need proactive attention?

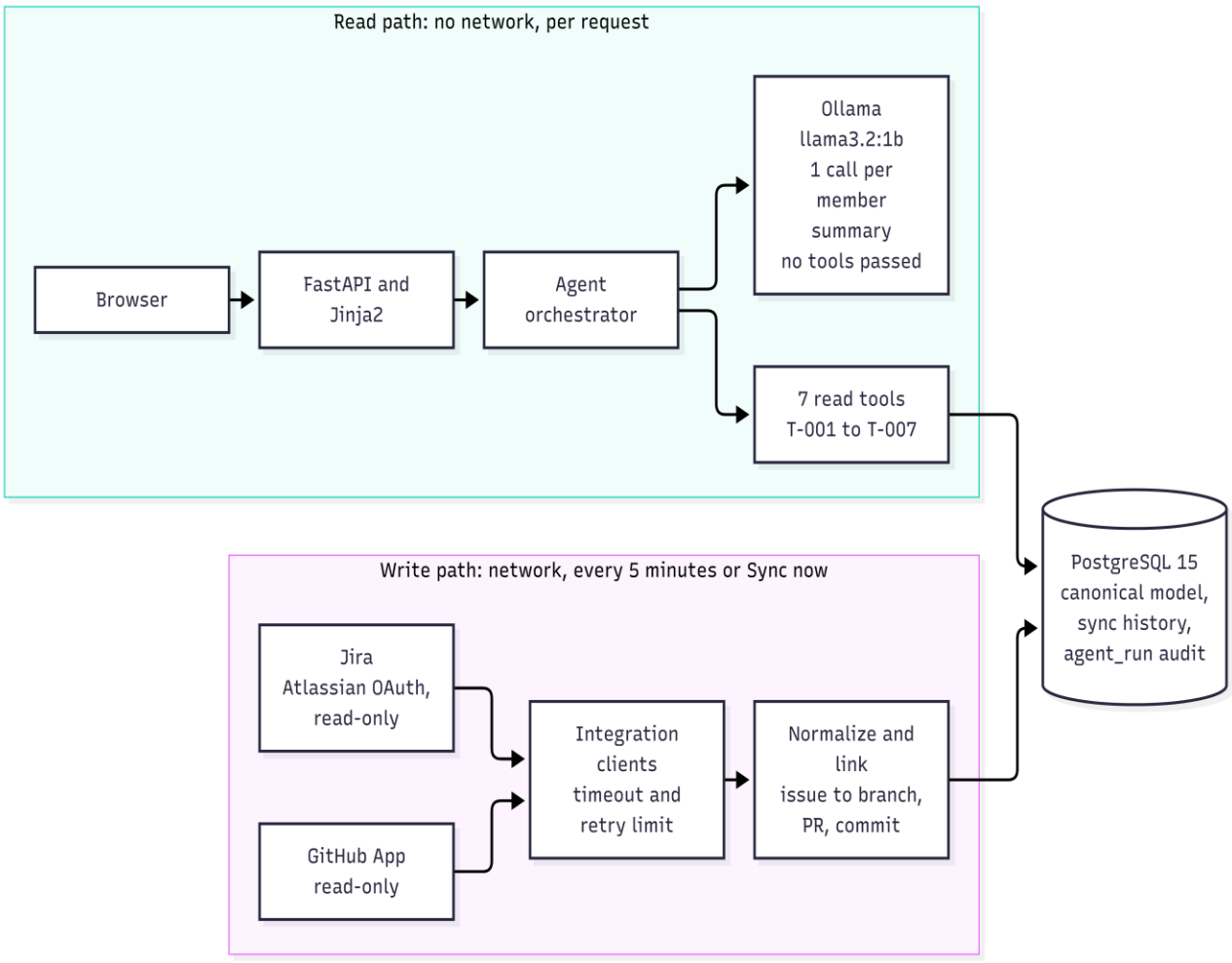
Every claim generated by headref is categorized and explicitly traced back to physical database source records:

- **FACT:** Backed by 1 exact source record (e.g., Jira issue assignment).
- **INFERENCE:** Derived from 2+ cross-source records (e.g., Jira ticket + active branch).
- **UNKNOWN:** Indeterminate state due to missing or unavailable source data.

2. Dual Data-Path Architecture

The foundational architectural mandate of headref (DEC-002) is the strict separation of the Write Path and Read Path. Agent-facing tools are physically prohibited from making outbound HTTP calls during user queries, guaranteeing that the agent operates deterministically over local PostgreSQL data.

2.1 Write Path and Read Path Overview



Data Path	Execution Mechanism	Core Responsibility & Key Rules
Write Path (Scheduled)	Background Cron / Scheduler (Dev 1 Lead)	Calls GitHub & Jira REST APIs, handles network retries, normalizes data into 17 canonical tables, maps identities, and builds ticket-code links. Strictly off the request path.
Read Path (Per Request)	FastAPI / Agent Orchestrator (Dev 2)	Queries PostgreSQL in read-only mode with zero external network calls. Runs 6 deterministic pipeline stages and invokes the local LLM once for narrative generation.

2.2 Six-Stage Deterministic Agent Pipeline

Every user question follows a rigid 6-stage execution pipeline designed for complete safety and auditability:

- **Stage 1 - S1 Plan (Code):** Selects a fixed retrieval plan based on question type (current_work, blockers, risks).
- **Stage 2 - S2 Retrieve (Code):** Executes T-007 (Source Health) first. If required sources are unavailable, the required-source gate closes early, returning an honest UNKNOWN state without querying the LLM.
- **Stage 3 - S3 Build Evidence (Code):** Deduplicates, ranks, sanitizes, and assigns stable sequential IDs (ev_1, ev_2) to source excerpts.
- **Stage 4 - S4 Reason (Code + LLM):** S4a runs deterministic findings rules in pure Python. S4b invokes the local Ollama LLM exactly once to write a 1-2 sentence prose summary.
- **Stage 5 - S5 Validate (Code):** Validates model output against strict rules (hallucinated entity check, forbidden judgment language, schema bounds, refusal detection), falling back to deterministic summaries if violated.
- **Stage 6 - S6 Respond (Code):** Caches results by evidence hash and persists the complete audit run into agent_run table.

3. Detailed Subsystem & Tool Designs

System functionality is partitioned across Developer 1's Write Path / Ingestion Engine and Developer 2's Read Path Tools. All interfaces utilize strict Pydantic v2 schemas for runtime input/output validation.

3.1 Write Path Services & Ingestion Engines (Developer 1 - Group Leader)

Service / Engine	Purpose and implementation	Sample output / contract
GitHub Client & Normalizer	Reads repos, branches, commits, PRs, reviews, and CI status via fine-grained App tokens. Normalizes state, strips URLs, and caps excerpts at 500 characters.	`{"number": 6, "state": "open", "checks_state": "failing"}`
Jira Client & Normalizer	Executes JQL searches over Atlassian OAuth 3LO endpoints. Extracts impediment flags, priority, due dates, and blocking links.	`{"external_id": "HRS-7", "status": "blocked", "is_flagged": true}`
Identity Resolution Graph	Maps external GitHub logins and Jira accountIDs to one internal app_user using verified manual links. Unmapped accounts enter the unmatched_entity queue.	`{"user_id": 7, "matched_via": "manual", "confidence": "HIGH"}`
Work Item Link Builder	Scans branches, PR titles, PR bodies, and commit messages for ticket keys matching (HRS|AUTH)-\d+. Assigns link confidence using the DEC-009 confidence matrix.	`{"work_item": "HRS-6", "target": "PR#4", "confidence": "HIGH"}`

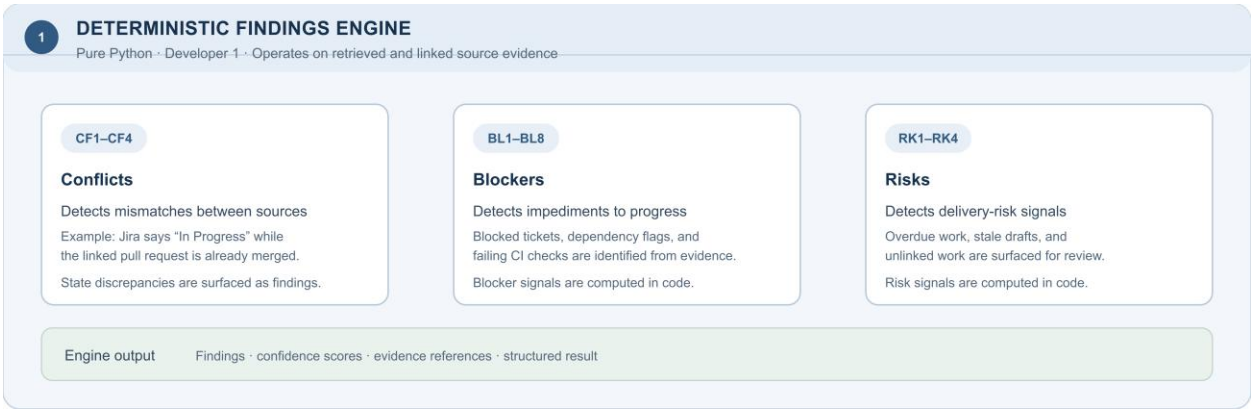
3.2 Agent-Facing Read Tools T-001 - T-007 (Developer 2)

Tool name	Purpose and operational scope (DB only)	Pydantic output sample
T-001 get_team_members	Retrieves active team members with verified linked integration accounts.	`{"user_id": 7, "github_handle": "cclarke642"}`
T-002 get_assigned_work_items	Fetches assigned Jira items within time windows with dependency status.	`{"external_id": "HRS-6", "blocked_by": ["HRS-7"]}`
T-003 get_pull_requests	Retrieves normalized GitHub PRs, draft status, review states, and CI checks.	`{"pr_number": 12, "review_state": "changes_requested"}`
T-004 get_commits	Fetches commits authored in the requested window; sets truncated=true if results are capped.	`{"commits": [{"sha": "6ed27c0"}], "truncated": false}`
T-005 get_reviews	Retrieves code review decisions given or received by the team member.	`{"state": "approved", "pull_request_number": 4}`
T-006 get_work_item_links	Queries pre-computed correlation links connecting Jira tickets to code.	`{"work_item_id": "HRS-6", "target_type": "pull_request"}`
T-007 get_source_health	Evaluates per-source freshness (fresh, stale, unavailable). Called first on every run.	`{"source": "jira", "state": "unavailable", "error": "AUTH_FAILED"}`

4. Shared Findings Engine & LLM Prompt Design

To eliminate LLM hallucinations on critical claims, headref enforces DEC-018: All findings, confidence scores, blockers, and risks are computed in pure Python code (Developer 1). The local Ollama LLM (llama3.2:1b) is called exactly once in Stage S4b with zero tools attached, acting solely as a natural language prose generator.

4.1 Deterministic Findings Engine



4.2 Prose Narrative Generation



5. Testing, Security & Hardening

headref was subjected to rigorous dual-layer testing comprising standard pytest suites (620+ unit/integration tests) and an automated 12-scenario evaluation suite (EV-01 through EV-12).

5.1 Write Path Security Gate (P7-001, Developer 1):

Implemented an AST static scan verifying zero outbound HTTP clients under app/tools/, no write calls to upstream APIs, and complete token redaction across log lines, exception details, and agent_run database rows. Network escape guards block unauthorized outbound sockets during unit test execution.

5.2 Read Path Evaluation Harness (Developer 2):

12 adverse scenarios tested agent honesty under stale sources, total API outages, conflicting cross-source state, unmatched commit authors, prompt injection inside PR bodies, cross-workspace isolation breaches, and malformed model outputs. All 12 passed.

6. Submission Deliverables & Individual Contribution Matrix

6.1 Project Deliverables

1. **GitHub Repository:** https://github.com/c-clarke-group-projects/headref.git
2. **Demo Video:** headref_demo_mp4

6.2 Individual Contribution

headref Individual Contribution Matrix

Project module ownership and contributions · Part 1 of 2

Project module	W. A. Shamika Keshan Leader / Developer 1	K. Isiwara Didulani Developer 2
Write Path & Ingestion Engine	Primary ownership: Designed and built GitHub/Jira REST clients, JQL search pagination, Atlassian OAuth 3LO, normalization schemas, and the workspace background sync scheduler (WO-W11).	Reviewed client schemas and verified sync status endpoints.
Identity Graph & Link Engine	Primary ownership: Created the identity resolution graph, manual/inferred link engine, unmatched entity queue, and regex ticket correlation engine (CF1–CF4).	Verified identity mapping against test fixtures.
Findings Engine & Rules	Primary ownership: Authored deterministic conflict, blocker (BL1–BL8), and risk (RK1–RK4) rules; confidence scoring engine; and fallback summaries (P4-002/3/4/8).	Integrated the findings engine into the Stage S4a pipeline.

Contribution labels and identifiers follow the supplied project report.

headref · Contributions

headref Individual Contribution Matrix

Project module ownership and contributions · Part 2 of 2

Project module	W. A. Shamika Keshan Leader / Developer 1	K. Isiwara Didulani Developer 2
Agent Pipeline & Read Tools	Built DB-only tools T-001–T-007 (P3-002/3) and added the T-007 source health gate.	Primary ownership: Built the S1 planner, S2 retrieval, S3 evidence builder, S4b Ollama narrative call, S5 validator, and S6 cache.
API, UI & Multi-Tenancy	Primary ownership: Built multi-tenant workspace architecture (DEC-021), permission guards, settings, invitations, dark/light theme, and UI SVG assets.	Built FastAPI route handlers, Jinja2 page templates, the JavaScript evidence drawer, and freshness banners.
Testing, Security & Setup	Primary ownership: Built the Write Path test suite, AST security scan, secret sanitizer, network escape guard, Docker Compose setup, and captured sandbox test data.	Built Read Path unit tests, the 12-scenario evaluation harness (EV-01–EV-12), and E2E integration test.

Contribution labels and identifiers follow the supplied project report.

headref · Contributions

Appendix: AI Coding Assistant Disclosure

In accordance with assignment guidelines, both developers utilized an AI coding assistant (Claude) for drafting initial boilerplate, writing unit tests, and debugging. Every generated block was manually inspected, refactored, and verified by human developers. Architectural decisions (DEC-001 - DEC-026), security gate policies, and domain logic were authored exclusively by the human team.